

Database Management System

B.Tech. II Year II Semester

UNIT - I

Database System Applications: A Historical Perspective, File Systems versus a DBMS, the Data Model, Levels of Abstraction in a DBMS, Data Independence, Structure of a DBMS

Introduction to Database Design: Database Design and ER Diagrams, Entities, Attributes, and Entity Sets, Relationships and Relationship Sets, Additional Features of the ER Model, Conceptual Design with the ER Model

UNIT - II

Introduction to the Relational Model: Integrity constraint over relations, enforcing integrity constraints, querying relational data, logical data base design, introduction to views, destroying/altering tables and views.

Relational Algebra, Tuple relational Calculus, Domain relational calculus.

UNIT - III

SQL: QUERIES, CONSTRAINTS, TRIGGERS: form of basic SQL query, UNION, INTERSECT, and EXCEPT, Nested Queries, aggregation operators, NULL values, complex integrity constraints in SQL, triggers and active data bases.

Schema Refinement: Problems caused by redundancy, decompositions, problems related to decomposition, reasoning about functional dependencies, FIRST, SECOND, THIRD normal forms, BCNF, lossless join decomposition, multi-valued dependencies, FOURTH normal form, FIFTH normal form.

UNIT - IV

Transaction Concept, Transaction State, Implementation of Atomicity and Durability, Concurrent Executions, Serializability, Recoverability, Implementation of Isolation, Testing for serializability, Lock Based Protocols, Timestamp Based Protocols, Validation- Based Protocols, Multiple Granularity, Recovery and Atomicity, Log-Based Recovery, Recovery with Concurrent Transactions.

UNIT - V

Data on External Storage, File Organization and Indexing, Cluster Indexes, Primary and Secondary Indexes, Index data Structures, Hash Based Indexing, Tree base Indexing, Comparison of File Organizations, Indexes and Performance Tuning, Intuitions for tree Indexes, Indexed Sequential Access Methods (ISAM), B+ Trees: A Dynamic Index Structure.

UNIT – III Course Overview:

INTRODUCTION TO SQL

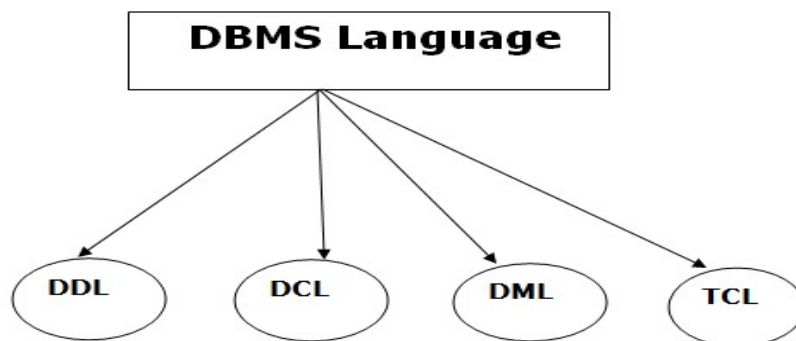
- DDL , DML, DCL & TCL Statements
- Nested Queries
- Triggers

SCHEMA REFINEMENT

- Functional Dependencies
- Normalization Concept
- 1NF, 2NF,3NF,BCNF,4NF,5NF

Query Language - Query languages are used to *read, update and store data in a database*. There are several such languages that can be used for this purpose; one of them is SQL (Structured Query Language).

Types of Query Language

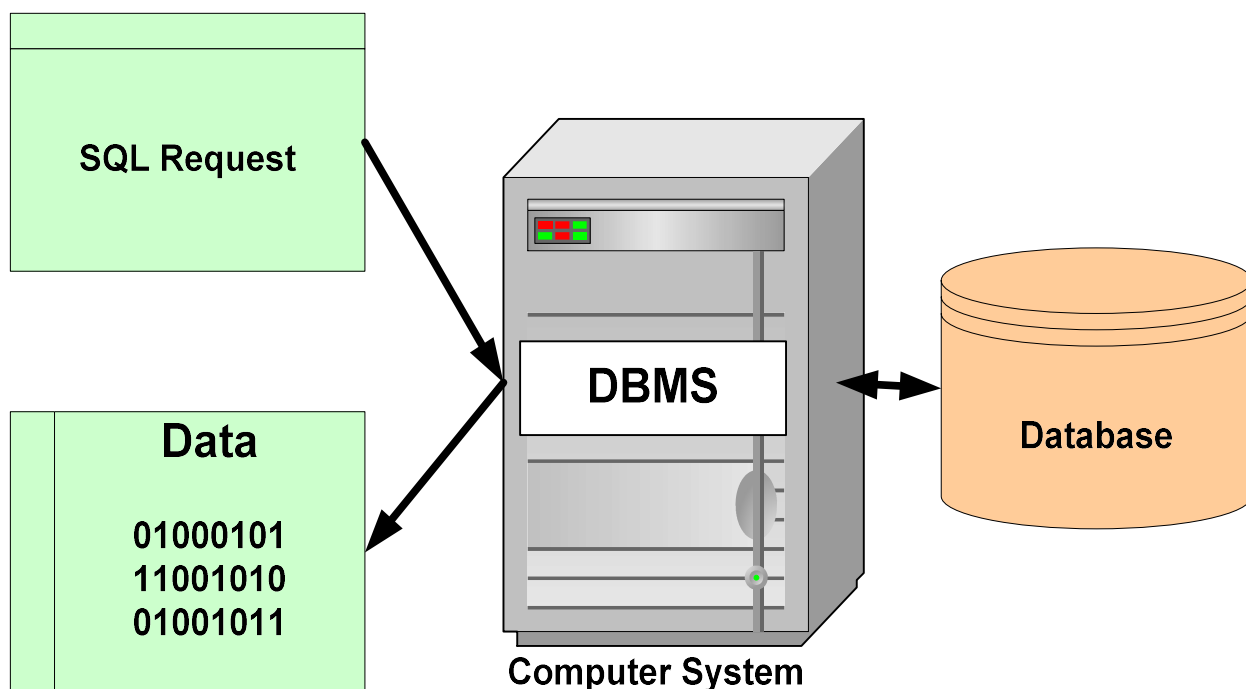


INTRODUCTION TO SQL

- SQL is used to make a request to retrieve data from a Database.

- The DBMS processes the SQL request, retrieves the requested data from the Database, and returns it.
- This process of requesting data from a Database and receiving back the results is called a Database Query and hence the name Structured Query Language.
- SQL is a language that all commercial RDBMS implementations understand.
- SQL is a non-procedural language
- We would be discussing SQL with respect to **Oracle & Mysql** syntax

Structured Query Language (SQL)



DATA DEFINITION LANGUAGE (DDL)

DDL is used for specifying the database schema. It is used for creating tables, schema, indexes, constraints etc. in database. The following are the operations that we can perform on database using DDL:

To create the database instance – **CREATE**

To alter the structure of database – **ALTER**

To drop database instances – **DROP**

To delete tables in a database instance – **TRUNCATE**

To rename database instances – **RENAME**

To drop objects from database such as tables – **DROP**

To Comment on the data dictionary – **Comment**

DATA MANIPULATION LANGUAGE (DML)

DML is used for accessing and manipulating data in a database. The following operations on database come under DML:

To read records from table(s) – **SELECT**

To insert record(s) into the table(s) – **INSERT**

Update the data in table(s) – **UPDATE**

Delete all the records from the table – **DELETE**

DATA CONTROL LANGUAGE (DCL)

DCL is used for granting and revoking user access on a database –

To grant access to user – **GRANT**

To revoke access from user – **REVOKE**

TRANSACTION CONTROL LANGUAGE (TCL)

The changes in the database that we made using DML commands are either performed or roll backed using TCL.

To persist the changes made by DML commands in database – **COMMIT**

To rollback the changes made to the database – ROLLBACK

DATA TYPES

- Number
- Char
- Varchar2
- Long
- Date

SQL supports various data types

Integers & Decimal numbers--- NUMBER, INTEGER.

Number is an oracle data type. Integer is an ANSI data type. Integer is equivalent of NUMBER (38). The syntax for NUMBER is NUMBER(P,S) p is the precision and s is the scale. P can range from 1 to 38 and s from -84 to 127

Floating point numbers---- FLOAT

- Number
- Char
- Varchar2
- Long
- Date

SQL supports various data types

Integers & Decimal numbers--- NUMBER, INTEGER .

Number is an oracle data type. Integer is an ANSI data type. Integer is equivalent of NUMBER(38). The syntax for NUMBER is NUMBER(P,S) p is the precision and s is the scale. P can range from 1 to 38 and s from -84 to 127

Floating point numbers---- FLOAT

Fixed length character strings---- CHAR [<size>]

The CHAR data type is fixed length, with the maximum size of the column specified in parentheses. Specifying the size is optional, and the default size is 1 byte. The maximum allowed size in a CHAR data type column is 2000 bytes.

Variable length character strings --- VARCHAR2 (<size>)

A maximum size for the column should be defined. Unlike CHAR columns, VARCHAR2 columns are not blank-padded with trailing spaces if the column value is shorter than its maximum specified length. The range of values allowed for size is from 1 to 4000 bytes.

Dates-----DATE

The DATE data type stores date and time information. You can store the dates from January 1, 4712 BC to December 31, 9999 AD.

LONG---- LONG columns can store up to 2GB of data. There can be only one LONG column in the table definition.

NULL

- Missing/unknown/inapplicable data represented as a NULL value
- NULL is not a data value. It is just an indicator that the value is unknown

OPERATORS

Arithmetic operators: +, -, *, /

Logical operators: AND, OR, NOT

Relational operators: =, <=, >=, < >, < , >

The Arithmetic operators are used to calculate something like given in the example below:

Select * from employee where sal * 1.1 > 1000 ;

The logical operators are used to combine conditions like:

Select * from employee where (sal > 1000 AND age > 25);

The above two examples also illustrate the use of relational operators.

SQL-DATA DEFINITION LANGUAGE

DDL Commands are

- CREATE
- ALTER
- DROP
- TRUNCATE

SQL - CREATE DATABASE

Syntax:

CREATE DATABASE databakiran*me*;

The CREATE DATABASE statement is used to create a new SQL database. i.e it creates the instance

Of the database schema in Database.

Example:

CREATE DATABASE college;

Once a database is created,

we can check it in the list of databases

with the following SQL command:

SHOW DATABASES

SQL – USE DATABASE

Syntax:

USE databakiran*me*

USE databakiran*me*. The **USE** statement tells **MySQL** to **use** the named **database** as the default (current) **database** for subsequent statements. This statement requires some privilege for the **database** or some object within it.

Example:

USE college;

SQL – DROP & BACKUP DATABASE

DROP DATABASE Statement - Statement is used to drop an existing SQL database.

Syntax:

DROP DATABASE *databakiranme*;

Example:

DROP DATABASE college;

BACKUP DATABASE Statement - Statement is used in SQL Server to create a full back up of an existing SQL database.

Syntax:

```
BACKUP DATABASE databakiranme TO DISK = 'filepath';
```

SQL - CREATE TABLE

Syntax:

CREATE TABLE *tablename*

```
(  
    column_name data_type constraints, ...  
);
```

Used to create a table by defining its structure, the data type and name of the various columns, the relationships with columns of other tables etc. Table names are used to identify each table and can be 30 characters long. The only special characters allowed in the table name are the dollar sign(\$), the underscore (_), and the pound sign (#).

Use DESCRIBE or DESC (SQL *Plus) command to list all of the columns in the table, along with their data type, size, nullity, and order.

The syntax is DESCRIBE <table name>

Example:

```
CREATE TABLE student(sno int(5), sname varchar(20), branch varchar(10));
```

```
DESC student;
```

```
SHOW TABLES;
```

SQL - ALTER TABLE

Syntax:

```
ALTER TABLE tablename (ADD/MODIFY/DROP column_name)
```

```
ALTER TABLE student add(mobile numeric(10));
```

```
ALTER TABLE student modify sno numeric(5);
```

```
ALTER TABLE student drop mobile;
```

SQL - DROP TABLE

- **DROP TABLE** - The DROP TABLE statement is used to drop an existing table in a database.
- Deletes table structure
- Cannot be recovered
- Use with caution

Syntax

```
DROP TABLE table_name;
```

Ex:

```
DROP TABLE student;
```

SQL - TRUNCATE TABLE

- **TRUNCATE TABLE** - Statement is used to delete the data inside a table, but not the table itself.

Syntax

- **TRUNCATE TABLE** *table_name*;

Example

```
TRUNCATE TABLE student;
```

SQL Create Constraints

- Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement.
- Constraints can be **Column level** or **Table level**. **Column level** constraints apply to a column, and **Table level** constraints apply to the whole table.
- **Syntax**
- **CREATE TABLE** *table_name* (*column1 datatype constraint, column2 datatype constraint, column3 datatype constraint,*);

Types of Constraints

- Primary Key Constraint
- Foreign Key Constraint
- Unique Constraint
- Check Constraint
- Not Null Constraint

SQL NOT NULL Constraints

By default, a column can hold NULL values. The NOT NULL constraint enforces a column to NOT accept NULL values.

Example:

```
CREATE TABLE student1(sno numeric(5) NOT NULL, sname varchar(20), branch varchar(10) );
```

```
INSERT INTO student1 VALUES(NULL,'chary',NULL); // Error
```

SQL UNIQUE Constraints

The UNIQUE constraint ensures that all values in a column are different. Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns. A PRIMARY KEY constraint automatically has a UNIQUE constraint. However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

Example:

```
CREATE TABLE student1(sno numeric(5) UNIQUE, sname varchar(20), branch varchar(10) NOT NULL);
```

```
INSERT INTO student1 VALUES(1,'chary','CSE');
```

```
INSERT INTO student1 VALUES(1,'chary','CSE');//Error
```

SQL PRIMARY KEY Constraints

The PRIMARY KEY constraint uniquely identifies each record in a table. Primary keys must contain UNIQUE values, and cannot contain NULL values. A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields)

Example:

```
CREATE TABLE student1(sno numeric(5) PRIMARY KEY, sname varchar(20), branch varchar(10));
```

```
INSERT INTO student1 VALUES(1,'chary','CSE');
```

```
INSERT INTO student1 VALUES(1,'chary','CSE');//Error
```

SQL CHECK Constraints

The CHECK constraint is used to limit the value range that can be placed in a column. If you define a CHECK constraint on a column it will allow only certain values for this column.

Example:

```
CREATE TABLE student1(sno numeric(5) PRIMARY KEY, sname varchar(20), branch varchar(10), CHECK (SNO>100));
```

```
INSERT INTO student1 VALUES(100,'chary','CSE');
```

```
INSERT INTO student1 VALUES(1,'chary','CSE');//Error
```

SQL DEFAULT Constraints

The DEFAULT constraint is used to set a default value for a column. The default value will be added to all new records, if no other value is specified

Example:

```
CREATE TABLE student1(sno numeric(5), sname varchar(20), branch varchar(10) DEFAULT 'CSE');
```

```
INSERT INTO student1(SNO,SNAME)VALUES(112,'chary');
```

```
INSERT INTO student1(SNO,SNAME)VALUES(100,'chary');
```

O/P: 112charyCSE

100charyCSE

SQL FOREIGN KEY Constraints

A **FOREIGN KEY** is a field (or collection of fields) in one table, that refers to the [PRIMARY KEY](#) in another table. The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table

Example:

```
CREATE TABLE DEPT(deptid numeric(4) primary key, dname varchar(10));
```

```
CREATE TABLE EMP1(eid numeric(4) primary key, ename varchar(10), deptid numeric(4), foreign  
key(deptid) references DEPT(deptid));
```

```
INSERT INTO DEPT values(10,'sales');
```

```
INSERT INTO DEPT values(20,'admin');
```

```
INSERT INTO EMP1 values(1,'a',10);
```

INSERT INTO EMP1 values(2,'a',30);//Error

DELETE from DEPT where deptid=10;//Error

SQL-Data Manipulation Language

DML Commands are

- INSERT
- SELECT
- UPDATE
- DELETE

SQL - INSERT INTO

The INSERT INTO statement is used to insert new records in a table

Syntax

It is possible to write the INSERT INTO statement in two ways:

1. Specify both the column names and the values to be inserted:

INSERT INTO *table_name* (*column1, column2, column3, ...*) VALUES (*value1, value2, ...*);

Example:

INSERT INTO student1(sno,sname) values(2,'kiran');

2. If you are adding values for all the columns of the table.(Must follow order of the columns

INSERT INTO *table_name* VALUES (*value1, value2, value3, ...*);

Example:

```
INSERT INTO student1 values(1,'kiran' ,'cse');
```

SQL SELECT Statement

The SELECT statement is used to select data from a database.

The data returned is stored in a result table, called the result-set.

SELECT Syntax

```
SELECT column1, column2, ... FROM table_name;
```

Here, *column1, column2, ...* are the field names of the table you want to select data from. If you want to select all the fields available in the table, use the following syntax:

```
SELECT * FROM table_name;
```

Example

```
SELECT CustomerName, City FROM Customers
```

```
SELECT * FROM Customers;
```

SQL SELECT DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values. Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.

Syntax

```
SELECT DISTINCT column1, column2, ... FROM table_name;
```

Example:


```
SELECT DISTINCT sname FROM student1;
```

SQL WHERE Clause

The WHERE clause is used to filter records. It is used to extract only those records that fulfill a specified condition.

WHERE Syntax

```
SELECT column1, column2, ... FROM table_name WHERE condition;
```

Note: The WHERE clause is not only used in SELECT statements, it is also used in UPDATE, DELETE, etc.!

Example:

```
SELECT * FROM student1 WHERE branch='cse';
```

Operators in The WHERE Clause

The following operators can be used in the **WHERE** clause:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

SQL AND, OR and NOT Operators

The SQL AND, OR and NOT Operators

The WHERE clause can be combined with AND, OR, and NOT operators.

The AND and OR operators are used to filter records based on more than one condition:

AND Syntax

```
SELECT column1, column2, ... FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

OR Syntax

```
SELECT column1, column2, ... FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT Syntax

```
SELECT column1, column2, ... FROM table_name WHERE NOT condition;
```

SQL ORDER BY Keyword

The **ORDER BY** keyword is used to sort the result-set in ascending or descending order.

The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

ORDER BY Syntax

```
SELECT column1, column2, ... FROM table_name ORDER BY column1, column2, ... ASC|DESC;
```

Example:

```
SELECT * FROM student1 ORDER BY sname;
```

```
SELECT * FROM student1 ORDER BY sname DESC;
```

```
SELECT * FROM student1 ORDER BY sname, branch;
```

```
SELECT * FROM student1 ORDER BY sname ASC, branch DESC;
```

SQL UPDATE Statement

The UPDATE statement is used to modify the existing records in a table.

UPDATE Syntax

```
UPDATE table_name SET column1 = value1, column2 = value2, ...WHERE condition;
```

Example

```
UPDATE student SET branch = 'cse', WHERE sno = 1;
```

Note: While using UPDATE command, don't forget to use WHERE condition. If we omit, all the records will get updated.

SQL DELETE Statement

The **DELETE** statement is used to delete existing records in a table.

DELETE Syntax

```
DELETE FROM table_name WHERE condition;
```

Example

```
DELETE FROM student WHERE sno = 1;
```

Note: While using DELETE command, don't forget to use WHERE condition. If we omit, all the records will get deleted.

Delete All Records

```
DELETE FROM table_name;
```

Difference Between Delete and Truncate

DELETE	TRUNCATE
Data can be recovered	Data cannot be recovered.
DML statement	DDL statement
DELETE does not release the memory occupied by the records of the table	TRUNCATE releases the memory occupied by the records of the table

SQL MIN() and MAX() Functions

The **MIN()** function returns the smallest value of the selected column.

The **MAX()** function returns the largest value of the selected column.

MIN() Syntax

```
SELECT MIN(column_name) FROM table_name WHERE condition;
```

MAX() Syntax

```
SELECT MAX(column_name) FROM table_name WHERE condition;
```

Example:

```
SELECT MIN(sal) FROM emp
```

```
SELECT MAX(sal) FROM emp
```

SQL COUNT(), AVG() and SUM() Functions

The **COUNT()** function returns the number of rows that matches a specified criterion.

COUNT() Syntax

```
SELECT COUNT(column_name) FROM table_name WHERE condition;
```

The **AVG()** function returns the average value of a numeric column.

AVG() Syntax

```
SELECT AVG(column_name) FROM table_name WHERE condition;
```

The **SUM()** function returns the total sum of a numeric column.

SUM() Syntax

```
SELECT SUM(column_name) FROM table_name WHERE condition;
```

SQL IN Operator

The SQL IN Operator

The IN operator allows you to specify multiple values in a WHERE clause.

The IN operator is a shorthand for multiple OR conditions.

IN Syntax

```
SELECT column_name(s) FROM table_name WHERE column_name IN (value1, value2, ...);
```

or:

`SELECT column_name(s) FROM table_name WHERE column_name IN (SELECT STATEMENT);`

Example:

`SELECT * FROM student1 WHERE branch IN ('cse', 'ece')`

SQL BETWEEN Operator

The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.

The **BETWEEN** operator is inclusive: begin and end values are included.

BETWEEN Syntax

`SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2;`

Example

`SELECT * FROM emp WHERE deptid BETWEEN 10 AND 50;`

SQL Aliases

SQL aliases are used to give a table, or a column in a table, a temporary name.

Aliases are often used to make column names more readable. An alias only exists for the duration of that query. An alias is created with the **AS** keyword.

Alias Column Syntax

`SELECT column_name AS alias_name FROM table_name;`

Example

```
SELECT CustomerID AS ID, CustomerName AS Customer FROM Customers;
```

Alias Table Syntax

```
SELECT column_name(s) FROM table_name AS alias_name;
```

Example

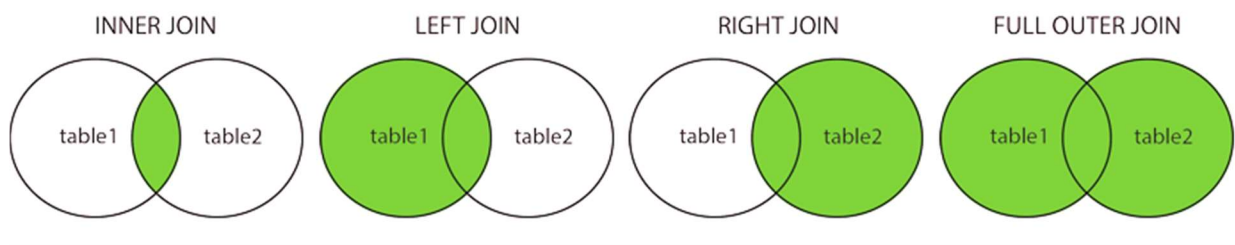
```
SELECT o.OrderID, o.OrderDate, c.CustomerName FROM Customers AS c, Orders AS o  
WHERE c.CustomerName='Around the Horn' AND c.CustomerID=o.CustomerID;
```

SQL JOINS

As the name shows, JOIN means to combine something. In case of SQL, JOIN means "to combine two or more tables". In SQL, JOIN clause is used to combine the records from two or more tables in a database.

Types of SQL JOINS

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN



Sample Tables

EMPLOYEE TABLE

EMP_ID	EMP_NAME	CITY	SALARY	AGE
1	Angelina	Chicago	200000	30
2	Robert	Austin	300000	26
3	Christian	Denver	100000	42
4	Kristen	Washington	500000	29
5	Russell	Los angels	200000	36
6	Marry	Canada	600000	48

PROJECT TABLE

PROJECT_NO	EMP_ID	DEPARTMENT
101	1	Testing
102	2	Development
103	3	Designing
104	4	Development

1. INNER JOIN

In SQL, INNER JOIN selects records that have matching values in both tables as long as the condition is satisfied. It returns the combination of all rows from both the tables where the condition satisfies.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....  
FROM table1  
INNER JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Example:

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT  
FROM EMPLOYEE INNER JOIN PROJECT  
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development

2. LEFT JOIN

The SQL left join returns all the values from left table and the matching values from the right table. If there is no matching join value, it will return NULL.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....  
FROM table1  
LEFT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Example:

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT
FROM EMPLOYEE
LEFT JOIN
PROJECT
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development
Russell	NULL
Marry	NULL

3. RIGHT JOIN

In SQL, RIGHT JOIN returns all the values from the values from the rows of right table and the matched values from the left table. If there is no matching in both tables, it will return NULL.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....
FROM table1 RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```

Example:

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT
FROM EMPLOYEE RIGHT JOIN PROJECT
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development

4. FULL JOIN

In SQL, FULL JOIN is the result of a combination of both left and right outer join. Join tables have all the records from both tables. It puts NULL on the place of matches not found.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....  
FROM table1 FULL JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Example:

```
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT  
FROM EMPLOYEE FULL JOIN PROJECT  
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
```

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development
Russell	NULL
Marry	NULL

SQL Set Operations

The SQL Set operation is used to combine the two or more SQL SELECT statements.

Types of Set Operations

1. Union
2. UnionAll
3. Intersect
4. Minus



1. Union

- The SQL Union operation is used to combine the result of two or more SQL SELECT queries.
- In the union operation, all the number of datatype and columns must be same in both the tables on which UNION operation is being applied.
- The union operation eliminates the duplicate rows from its resultset.

Syntax

```
SELECT column_name FROM table1  
UNION  
SELECT column_name FROM table2;
```

Example:

The First table

ID	NAME
1	Jack
2	Harry
3	Jackson

The Second table

ID	NAME
3	Jackson
4	Stephan
5	David

Union SQL query will be:

```
SELECT * FROM First
UNION
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
4	Stephan
5	David

2. Union All

Union All operation is equal to the Union operation. It returns the set without removing duplication and sorting the data.

Syntax:

```
SELECT column_name FROM table1
UNION ALL
SELECT column_name FROM table2;
```

Union All query will be like:

```
SELECT * FROM First
UNION ALL
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
3	Jackson
4	Stephan
5	David

3. Intersect

- It is used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
- In the Intersect operation, the number of datatype and columns must be the same.
- It has no duplicates and it arranges the data in ascending order by default.

Syntax

```
SELECT column_name FROM table1
INTERSECT
SELECT column_name FROM table2;
```

Example:

Using the above First and Second table.

Intersect query will be:

```
SELECT * FROM First
INTERSECT
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
3	Jackson

4. Minus

- It combines the result of two SELECT statements. Minus operator is used to display the rows which are present in the first query but absent in the second query.
- It has no duplicates and data arranged in ascending order by default.

Syntax:

```
SELECT column_name FROM table1  
MINUS  
SELECT column_name FROM table2;
```

Example

Using the above First and Second table.

Minus query will be:

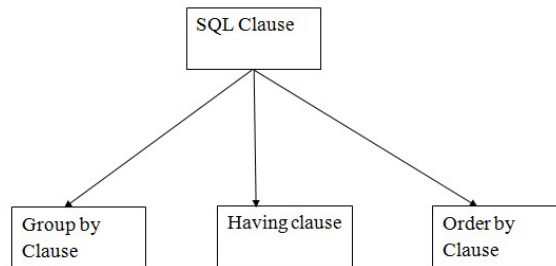
```
SELECT * FROM First  
MINUS  
SELECT * FROM Second;
```

The resultset table will look like:

ID	NAME
1	Jack
2	Harry

SQL Clauses

The following are the various SQL clauses:



1. GROUP BY

- SQL GROUP BY statement is used to arrange identical data into groups. The GROUP BY statement is used with the SQL SELECT statement.
- The GROUP BY statement follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.
- The GROUP BY statement is used with aggregation function.

Syntax

SELECT column
FROM table_name
WHERE conditions
GROUP BY column
ORDER BY column

Sample table:

PRODUCT_MAST

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40
Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30
Item9	Com2	2	25	50

Item10	Com3	4	30	120
--------	------	---	----	-----

Example:

```
SELECT COMPANY, COUNT(*)
FROM PRODUCT_MAST
GROUP BY COMPANY;
```

Output:

```
Com1 5
Com2 3
Com3 2
```

2. HAVING

- HAVING clause is used to specify a search condition for a group or an aggregate.
- Having is used in a GROUP BY clause. If you are not using GROUP BY clause then you can use HAVING function like a WHERE clause.

Syntax:

```
SELECT column1, column2
FROM table_name
WHERE conditions
GROUP BY column1, column2
HAVING conditions
ORDER BY column1, column2;
```

Example:

```
SELECT COMPANY, COUNT(*)
FROM PRODUCT_MAST
GROUP BY COMPANY
HAVING COUNT(*)>2;
```

Output:

```
Com1 5
Com2 3
```

3. ORDER BY

- The ORDER BY clause sorts the result-set in ascending or descending order.
- It sorts the records in ascending order by default. DESC keyword is used to sort the records in descending order.

Syntax:

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column1, column2... ASC|DESC;
```

Where

ASC: It is used to sort the result set in ascending order by expression.

DESC: It sorts the result set in descending order by expression.

Example: Sorting Results in Ascending Order

Table:

CUSTOMER

CUSTOMER_ID	NAME	ADDRESS
12	Kathrin	US
23	David	Bangkok
34	Alina	Dubai
45	John	UK
56	Harry	US

Enter the following SQL statement:

```
SELECT *
FROM CUSTOMER
ORDER BY NAME;
```

Output:

CUSTOMER_ID	NAME	ADDRESS
34	Alina	Dubai
23	David	Bangkok
56	Harry	US
45	John	UK
12	Kathrin	US

Example: Sorting Results in Descending Order

Using the above CUSTOMER table

```
SELECT *
FROM CUSTOMER
ORDER BY NAME DESC;
```

Output:

CUSTOMER_ID	NAME	ADDRESS
12	Kathrin	US
45	John	UK
56	Harry	US
23	David	Bangkok
34	Alina	Dubai

SQL Sub Query

A Subquery is a query within another SQL query and embedded within the WHERE clause.

Important Rule:

- A subquery can be placed in a number of SQL clauses like WHERE clause, FROM clause, HAVING clause.
- You can use Subquery with SELECT, UPDATE, INSERT, DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.
- A subquery is a query within another query. The outer query is known as the main query, and the inner query is known as a subquery.
- Subqueries are on the right side of the comparison operator.
- A subquery is enclosed in parentheses.
- In the Subquery, ORDER BY command cannot be used. But GROUP BY command can be used to perform the same function as ORDER BY command.

1. Subqueries with the Select Statement

SQL subqueries are most frequently used with the Select statement.

Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name expression operator  
( SELECT column_name from table_name WHERE ... );
```

Example

Consider the EMPLOYEE table have the following records:

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
4	Alina	29	UK	6500.00
5	Kathrin	34	Bangalore	8500.00
6	Harry	42	China	4500.00
7	Jackson	25	Mizoram	10000.00

The subquery with a SELECT statement will be:

```
SELECT *  
FROM EMPLOYEE  
WHERE ID IN (SELECT ID  
FROM EMPLOYEE  
WHERE SALARY > 4500);
```

This would produce the following result:

ID	NAME	AGE	ADDRESS	SALARY
4	Alina	29	UK	6500.00
5	Kathrin	34	Bangalore	8500.00
7	Jackson	25	Mizoram	10000.00

2. Subqueries with the INSERT Statement

- SQL subquery can also be used with the Insert statement. In the insert statement, data returned from the subquery is used to insert into another table.
- In the subquery, the selected data can be modified with any of the character, date functions.

Syntax:

```
INSERT INTO table_name (column1, column2, column3....)  
SELECT *  
FROM table_name  
WHERE VALUE OPERATOR
```

Example

Consider a table EMPLOYEE_BKP with similar as EMPLOYEE.

Now use the following syntax to copy the complete EMPLOYEE table into the EMPLOYEE_BKP table.

```
INSERT INTO EMPLOYEE_BKP  
SELECT * FROM EMPLOYEE  
WHERE ID IN (SELECT ID  
FROM EMPLOYEE);
```

3. Subqueries with the UPDATE Statement

The subquery of SQL can be used in conjunction with the Update statement. When a subquery is used with the Update statement, then either single or multiple columns in a table can be updated.

Syntax

```
UPDATE table
SET column_name = new_value
WHERE VALUE OPERATOR
(SELECT COLUMN_NAME
FROM TABLE_NAME
WHERE condition);
```

Example

Let's assume we have an EMPLOYEE_BKP table available which is backup of EMPLOYEE table. The given example updates the SALARY by .25 times in the EMPLOYEE table for all employee whose AGE is greater than or equal to 29.

```
UPDATE EMPLOYEE
SET SALARY = SALARY * 0.25
WHERE AGE IN (SELECT AGE FROM CUSTOMERS_BKP
WHERE AGE >= 29);
```

This would impact three rows, and finally, the EMPLOYEE table would have the following records.

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
4	Alina	29	UK	1625.00
5	Kathrin	34	Bangalore	2125.00
6	Harry	42	China	1125.00
7	Jackson	25	Mizoram	10000.00

4. Subqueries with the DELETE Statement

The subquery of SQL can be used in conjunction with the Delete statement just like any other statements mentioned above.

Syntax

```
DELETE FROM TABLE_NAME
```

```
WHERE VALUE OPERATOR  
(SELECT COLUMN_NAME  
FROM TABLE_NAME  
WHERE condition);
```

Example

Let's assume we have an EMPLOYEE_BKP table available which is backup of EMPLOYEE table. The given example deletes the records from the EMPLOYEE table for all EMPLOYEE whose AGE is greater than or equal to 29.

```
DELETE FROM EMPLOYEE  
WHERE AGE IN (SELECT AGE FROM EMPLOYEE_BKP  
WHERE AGE >= 29 );
```

This would impact three rows, and finally, the EMPLOYEE table would have the following records.

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
7	Jackson	25	Mizoram	10000.00

SQL EXISTS Operator

The EXISTS condition in SQL is used to check whether the result of a correlated nested query is empty (contains no tuples) or not. The result of EXISTS is a boolean value True or False. It can be used in a SELECT, UPDATE, INSERT or DELETE statement.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
(SELECT column_name(s)
FROM table_name
WHERE condition);
```

Examples: Consider the following two relation “Customers” and “Orders”.

Customers

customer_id	lname	fname	website
401	Singh	Dolly	abc.com
402	Chauhan	Anuj	def.com
403	Kumar	Niteesh	ghi.com
404	Gupta	Shubham	jkl.com
405	Walecha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

Orders

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
3	408	2017-01-18
4	404	2017-02-05

Queries

1. **Using EXISTS condition with SELECT statement** To fetch the first and last name of the customers who placed atleast one order.

```
SELECT fname, lname
```

```
FROM Customers
```

```
WHERE EXISTS (SELECT *
```

```
FROM Orders
```

```
WHERE Customers.customer_id = Orders.c_id);
```

Output:

fname	lname
Shubham	Gupta
Divya	Walecha
Rajiv	Mehta
Anand	Mehra

2. **Using NOT with EXISTS** Fetch last and first name of the customers who has not placed any order.

```
SELECT lname, fname
```

```
FROM Customers
```

```
WHERE NOT EXISTS (SELECT *
```

```
FROM Orders
```

```
WHERE Customers.customer_id = Orders.c_id);
```

Output:

lname	fname
Singh	Dolly
Chauhan	Anuj
Kumar	Niteesh
Jain	Sandeep

3. **Using EXISTS condition with DELETE statement** Delete the record of all the customer from Order Table whose last name is 'Mehra'.

DELETE

FROM Orders

WHERE EXISTS (SELECT *

FROM customers

WHERE Customers.customer_id = Orders.c_id

AND Customers.lname = 'Mehra');

SELECT * FROM Orders;

Output:

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
4	404	2017-02-05

4. **Using EXISTS condition with UPDATE statement** Update the lname as 'Kumari' of customer in Customer Table whose customer_id is 401.

UPDATE Customers

SET lname = 'Kumari'

WHERE EXISTS (SELECT *

FROM Customers

WHERE customer_id = 401);

SELECT * FROM Customers;

Output:

customer_id	lname	fname	website
401	Kumari	Dolly	abc.com
402	Chauhan	Anuj	def.com
403	Kumar	Niteesh	ghi.com
404	Gupta	Shubham	jkl.com
405	Walecha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

SQL ANY and ALL Operators

ALL & ANY are logical operators in SQL. They return boolean value as a result.

ALL

ALL operator is used to select all tuples of SELECT STATEMENT. It is also used to compare a value to every value in another value set or result from a subquery.

- The ALL operator returns TRUE if all of the subqueries values meet the condition. The ALL must be preceded by comparison operators and evaluates true if all of the subqueries values meet the condition.
- ALL is used with SELECT, WHERE, HAVING statement.

ALL with SELECT Statement:

Syntax:

```
SELECT ALL field_name
```

```
FROM table_name
```

```
WHERE condition(s);
```

ALL with WHERE or HAVING Statement:

Syntax:

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name comparison_operator ALL
```

```
(SELECT column_name
```

```
FROM table_name
```

```
WHERE condition(s));
```

Example: Consider the following Products Table and OrderDetails Table, **Products Table**

ProductID	ProductName	SupplierID	CategoryID	Price
1	Chais	1	1	18
2	Chang	1	1	19
3	Aniseed Syrup	1	2	10
4	Chef Anton's Cajun Seasoning	2	2	22
5	Chef Anton's Gumbo Mix	2	2	21
6	Boysenberry Spread	3	2	25
7	Organic Dried Pears	3	7	30
8	Northwoods Cranberry Sauce	3	2	40
9	Mishi Kobe Niku	4	6	97

OrderDetails Table

OrderDetailsID	OrderID	ProductID	Quantity
1	10248	1	12
2	10248	2	10
3	10248	3	15
4	10249	1	8
5	10249	4	4
6	10249	5	6
7	10250	3	5
8	10250	4	18
9	10251	5	2
10	10251	6	8
11	10252	7	9
12	10252	8	9
13	10250	9	20
14	10249	9	4

Queries

- Find the names of the all the product.

SELECT ALL ProductName

FROM Products

WHERE TRUE;

Output:

ProductName
Chais
Chang
Aniseed Syrup
Chef Anton's Cajun Seasoning
Chef Anton's Gumbo Mix
Boysenberry Spread
Organic Dried Pears
Northwoods Cranberry Sauce
Mishi Kobe Niku

- Find the name of the product if all the records in the OrderDetails has Quantity either equal to 6 or 2.

```
SELECT ProductName
```

```
FROM Products
```

```
WHERE ProductID = ALL (SELECT ProductId
```

```
FROM OrderDetails
```

```
WHERE Quantity = 6 OR Quantity = 2);
```

Output:

ProductName
Chef Anton's
Gumbo Mix

- Find the OrderID whose maximum Quantity among all product of that OrderID is greater than average quantity of all OrderID.

```
SELECT OrderID
```

```
FROM OrderDetails
```

```
GROUP BY OrderID
```

```
HAVING max(Quantity) > ALL (SELECT avg(Quantity)
```

```
FROM OrderDetails
```

```
GROUP BY OrderID);
```

Output:

OrderID
10248
10250

ANY

ANY compares a value to each value in a list or results from a query and evaluates to true if the result of an inner query contains at least one row.

- ANY return true if any of the subqueries values meet the condition.
- ANY must be preceded by comparison operators. **Syntax:**

```
SELECT column_name(s)
```

```
FROM table_name
```

```
WHERE column_name comparison_operator ANY
```

```
(SELECT column_name
```

```
FROM table_name
```

```
WHERE condition(s));
```

Queries

- Find the Distinct CategoryID of the products which have any record in OrderDetails Table.

```
SELECT DISTINCT CategoryID
```

```
FROM Products
```

```
WHERE ProductID = ANY (SELECT ProductID
```

```
FROM OrderDetails);
```

Output:

CategoryID
1
2
7
6

- Finds any records in the OrderDetails table that Quantity = 9.

```
SELECT ProductName
```

```
FROM Products
```

```
WHERE ProductID = ANY (SELECT ProductID
```

```
FROM OrderDetails
```

```
WHERE Quantity = 9);
```

Output:

ProductName
Organic Dried Pears
Northwoods Cranberry Sauce

SQL-CREATING NEW USER IN DATABASE

Syntax:

CREATE USER 'username' IDENTIFIED BY 'password';

Example:

CREATE USER 'kiran' IDENTIFIED BY 'kiran' ;

SQL-DATA CONTROL LANGUAGE

DCL Commands are

- **GRANT**-gives user's access privileges to the database.
- **REVOKE**-withdraw user's access privileges given by using the

GRANT command.

How To Grant Different User Permissions

Here is a short list of other common possible permissions that users can enjoy.

ALL PRIVILEGES- allow a MySQL user full access to a designated database (or if no database is selected, global access across the system)

CREATE- allows them to create new tables or databases

DROP- allows them to them to delete tables or databases

DELETE- allows them to delete rows from tables

INSERT- allows them to insert rows into tables

SELECT- allows them to use the SELECT command to read through databases

UPDATE- allow them to update table rows

GRANT OPTION- allows them to grant or remove other users' privileges

To provide a specific user with a permission, you can use this framework:

GRANT type_of_permission ON database_name.table_name TO 'username'@'localhost';

GRANT select ON mysql.temp TO 'kiran' ;

GRANT ALL PRIVILEGES on mysql.temp to kiran;

GRANT type_of_permission ON database_name.table_name TO 'username'@'localhost';

If you want to give them access to any database or to any table, make sure to put an asterisk (*) in the place of the database name or table name.

GRANT type_of_permission ON *.* TO 'username'@'localhost';

Each time you update or change a permission be sure to use the Flush Privileges command.

FLUSH PRIVILEGES;

If you need to revoke a permission, the structure is almost identical to granting it:

REVOKE type_of_permission ON database_name.table_name FROM 'username'@'localhost';

Note that when revoking permissions, the syntax requires that you use FROM, instead of TO as we used when granting permissions.

You can review a user's current permissions by running the following:

SHOW GRANTS FOR 'username'@'localhost';

Just as you can delete databases with DROP, you can use **DROP** to delete a user altogether:

DROP USER 'username'@'localhost';

SQL-Transaction Control Language

TCL COMMANDS ARE

COMMIT– Commits a Transaction.

ROLLBACK - Rollbacks a transaction in case of any error occurs.

SAVEPOINT–sets a save point within a transaction.

SET TRANSACTION–specify characteristics for the transaction.

SQL Views

- **CREATE VIEW** Statement
- **UPDATE VIEW** Statement
- **DROP VIEW** Statement

SQL CREATE View

CREATE VIEW Statement

In SQL, a view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database. You can add SQL statements and functions to a view and present the data as if the data were coming from one single table. A view is created with the CREATE VIEW statement.

CREATE VIEW Syntax

```
CREATE VIEW view_name AS SELECT column1, column2, ... FROM table_name  
WHERE condition;
```

Note: A view always shows up-to-date data! The database engine recreates the view, every time a user queries it.

```
CREATE VIEW emp1 AS SELECT * FROM emp WHERE sal>5000;
```

```
CREATE VIEW emp6 AS SELECT deptid from emp UNION SELECT deptid from DEPT;
```

SQL Updating View

A view can be updated with the **CREATE OR REPLACE VIEW** statement.

CREATE OR REPLACE VIEW Syntax

```
CREATE OR REPLACE VIEW view_name AS SELECT column1, column2, ...  
FROM table_name WHERE condition;
```

Example:

```
CREATE OR REPLACE VIEW emp1 AS SELECT eno,ename FROM emp WHERE sal>5000;
```

SQL Drop View

A view is deleted with the **DROP VIEW** statement.

DROP VIEW Syntax

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW EMP1;
```

What is Functional Dependency?

Functional Dependency (FD) is a constraint that determines the relation of one attribute to another attribute in a Database Management System (DBMS).

- Functional Dependency helps to maintain the quality of data in the database.
- It plays a important role to find the difference between good and bad database design.
- A functional dependency is denoted by an arrow " $\rightarrow$ ".
- The functional dependency of X on Y is represented by $X \rightarrow Y$.

Syntax:

Functional Dependency on any given table can be explained as,

$X \rightarrow Y$

Here, the left side of the arrow is identified as a **Determinant(X)**, while the right side of the arrow is identified as a **Dependent(Y)**.

X->Key Attribute, Y-> Non-Key Attribute

Example Database - Employee

Employee number	Employee Name	Salary	City
1	Rajesh	50000	Karimnagar
2	Raju	38000	Warangal
3	Raju	25000	Karimnagar
4	Rajesh	32000	Warangal
5	Krishna	25000	Hyderabad

In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc. By this, we can say that the city, Employee Name, and salary are functionally depended on Employee number

FUNCTIONAL DEPENDENCIES

A functional dependency is a relationship between two attributes. Typically, between the Primary Key and other non-key attributes within the table. For any relation R, attribute Y is functionally dependent on attribute X (usually the PK), if **for every valid instance of X, that value of X uniquely determines the value of Y.**

Definition: Let R be the relational schema and X, Y be the set of attributes over R.

T1, T2 be any tuples of R.

$X \rightarrow Y$ exists in relation R only

If **T1.X=T2.X**

then

T1.Y=T2.Y

If the Condition is true, then the dependency will exists. If the condition fails, then the dependency is not there.

TYPES OF FUNCTIONAL DEPENDENCIES IN DBMS:

Trivial functional dependency

Non-Trivial functional dependency

Multivalued functional dependency

Transitive functional dependency

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

1. TRIVIAL FUNCTIONAL DEPENDENCY

In **Trivial Functional Dependency**, a dependent is always a subset of the determinant.
i.e. If $X \rightarrow Y$ and **Y is the subset of X**, then it is called trivial functional dependency

For example,

Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set $\{\text{roll_no}, \text{name}\}$

Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

2. NON-TRIVIAL FUNCTIONAL DEPENDENCY

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant.
i.e. If $X \rightarrow Y$ and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

For example

Here, $\text{roll_no} \rightarrow \text{name}$ is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**

Similarly, $\{\text{roll_no}, \text{name}\} \rightarrow \text{age}$ is also a non-trivial functional dependency, since **age** is **not a subset of** $\{\text{roll_no}, \text{name}\}$

3. MULTIVALUED FUNCTIONAL DEPENDENCY

In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other**.

i.e. If $a \rightarrow \{b, c\}$ and there exists **no functional dependency** between **b** and **c**, then it is called a **Multivalued functional dependency**.

For example

Here, **roll_no** $\rightarrow$ {**name**, **age**} is a Multivalued functional dependency, since the dependents **name** & **age** are **not dependent**

on each other

(i.e. **name** $\rightarrow$ **age** or **age** $\rightarrow$ **name** doesn't exist !)

4. TRANSITIVE FUNCTIONAL DEPENDENCY

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If **a** $\rightarrow$ **b** & **b** $\rightarrow$ **c**, then according to axiom of transitivity, **a** $\rightarrow$ **c**. This is a **transitive functional dependency**

For example

Here, **enrol_no** $\rightarrow$ **dept** and **dept** $\rightarrow$ **building_no**, Hence, according to the axiom of transitivity, **enrol_no** $\rightarrow$ **building_no** is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

5. SINGLE VALUED FUNCTIONAL DEPENDENCY

The database is a collection of related information in which one information depends on another information. The information is either single-valued or multi-valued. For example, the name of the person or his / her date of birth is single-valued facts. However, the qualification of a person is a Multivalued fact.

A simple example of single value functional dependency is when A is the primary key of an entity (e.g. SID), and B is some single-valued attribute of the entity (e.g. SName)

Then $A \rightarrow B$ always holds FD.

CID	SID	Sname
C1	S1	A
C1	S2	A
C2	S1	A
C3	S1	A

$SID \rightarrow Sname$

S1 A

S1 A

S1 A

$Sname \rightarrow SID$ X

A S1

A S2

For every SID, there should be a unique name ($X \rightarrow Y$)

6. FULLY FUNCTIONAL DEPENDENCY

In a relation R, an attribute Q is said to be fully functional dependent on attribute P, if it is functionally dependent on P and not functionally dependent on any proper subset of P.

If $AD \rightarrow C$, is a functionally dependency, then we cannot remove A or D, i.e C is fully functionally dependent on AD. If we remove A or D, then it is not fully functionally dependency.

Example: $\{ENO, ENAME\} \rightarrow SAL$

7. PARTIAL FUNCTIONAL DEPENDENCY

A Functional Dependency in which one or more non-key attributes are functionally depending on the part of the primary key is called partial functional dependency.

If, $AD \rightarrow C$ then, $A \rightarrow C$, $D \rightarrow C$

Example: $\{ENO, DEPTID\} \rightarrow DNAME$, $ENO \rightarrow DNAME$, $DEPTID \rightarrow DNAME$

NORMALIZATION

Normalization is a process of organizing the data in Database to avoid Data Redundancy. Redundancy in relation may cause Insertion Anomaly, Update Anomaly & Deletion Anomaly.

Normalization is a systematic approach of decomposing tables to eliminate data redundancy(repetition) and undesirable characteristics like Insertion, Update and Deletion Anomalies.

Normalization is used for mainly two purposes,

- Eliminating redundant(useless) data.
- Ensuring data dependencies make sense i.e data is logically stored.

Problems Without Normalization

- If a table is not properly normalized and has data redundancy then it is difficult to handle and update the database, without facing data loss. Insertion, Update and Deletion Anomalies are very frequent if database is not Normalized.

ENO	ENAME	DEPTID	DNAME	DEPT_HOD
101	Rajesh	10	Admin	Mr. Vijary
102	Raju	20	Accounts	Mr. Santhosh
103	Raju	30	Store	Mr. Akash
104	Rajesh	20	Accounts	Mr. Santhosh
105	Krishna	30	Store	Mr. Akash

Employee Name is repeated for employee who is working for two departments – **Data Redundancy**

TYPES OF NORMAL FORMS

- First Normal Form
- Second Normal Form
- Third Normal Form
- BCNF
- Fourth Normal Form

1. FIRST NORMAL FORM (1NF)

For a table to be in the First Normal Form, it should follow the following 4 rules:

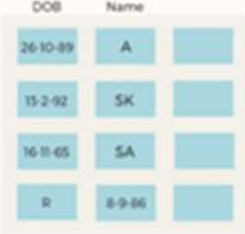
1. It should only have single(atomic) valued attributes/columns.
2. Values stored in a column should be of the same domain
3. All the columns in a table should have unique names.
4. And the order in which data is stored, does not matter.

RULE 1



- Each Column should contain atomic values.
- Entries like X, Y and W, X violate this rule.

RULE 2



- A Column should contain values that are of the same type.
- Do not inter-mix different types of values in any column.

RULE 3



- Each column should have a unique name.
- Same names leads to confusion at the time of data retrieval

RULE 4



- Order in which data is saved doesn't matter.
- Using SQL query, you can easily fetch data in any order from a table.

Example- Before Normalization

HTNO	NAME	SUBJECT
501	Rajesh	OS, CN
502	Ramesh	Java
503	Srinivas	C, C++

Example- After 1st Normalization

HTNO	NAME	SUBJECT
501	Rajesh	OS, CN
501	Rajesh	CN
502	Ramesh	Java
503	Srinivas	C
503	Srinivas	C++

2. SECOND NORMAL FORM (2NF)

For a table to be in the Second Normal Form, it must satisfy two conditions:

1. For a table to be in the Second Normal form, **it should be in the First Normal form and it should not have Partial Dependency.**
2. Partial Dependency exists, when for a composite primary key, any attribute in the table depends only on a part of the primary key and not on the complete primary key.
3. To remove Partial dependency, we can divide the table, remove the attribute which is causing partial dependency, and move it to some other table where it fits in well.

EMP TABLE

ENO	ENAME	DEPTID	DNAME	DEPT_HOD
101	Rajesh	10	Admin	Mr. Vijay
102	Raju	20	Accounts	Mr. Santhosh
103	Raju	30	Store	Mr. Akash
104	Rajesh	20	Accounts	Mr. Santhosh
105	Krishna	30	Store	Mr. Akash

EMP TABLE			DEPT TABLE		
ENO	ENAME	DEPTID	DEPTID	DNAME	DEPT_HOD
101	Rajesh	10	10	Admin	Mr. Vijay
102	Raju	20	20	Accounts	Mr. Santhosh
103	Raju	30	30	Store	Mr. Akash
104	Rajesh	20			
105	Krishna	30			

3. THIRD NORMAL FORM (3NF)

For a table to be in the Third Normal Form, it must satisfy two conditions:

For a table to be in the Third Normal form, **it should be in the Second Normal form and it should not have Transitive Dependency.**

1. It is in 2 NF
2. No Transitive Dependency for Non-Prime Attribute i.e. (**NPA->NPA**)

Advantage of removing Transitive Dependency

1. Amount of data duplication is reduced.
2. Data integrity achieved.

	</				

4. BOYCE-CODD NORMAL FORM (BCNF)

For a table to be in the BCNF, it must satisfy two conditions:

For a table to satisfy the Boyce-Codd Normal Form, it should satisfy the following two conditions:

1. It should be in the **Third Normal Form**.
2. And, for any dependency $A \rightarrow B$, A should be a **super key**.

It means, that for a dependency $A \rightarrow B$, A cannot be a **non-prime attribute**, if B is a **prime attribute**.

SID	SUBJECT	FACULTY
101	JAVA	Mr. Joy
101	C++	Mr. Joy
102	JAVA	Mr. Hari
103	PYTHON	Mr. Kiran
104	C	Mr. Joshi
KEY{SID+SUBJECT}, FACULTY->SUBJECT, NPA->PA , So not in BCNF		
After Decomposition		
FID	SUBJECT	FACULTY
1	JAVA	Mr. Joy
1	C++	Mr. Joy
2	JAVA	Mr. Hari
3	PYTHON	Mr. Kiran
4	C	Mr. Joshi
SID	FID	
101	1	
101	1	
102	2	
103	3	
104	4	

4. FOURTH NORMAL FORM (4NF)

For a table to be in the 4NF, it must satisfy two conditions:

For a table to satisfy the Fourth Normal Form, it should satisfy the following two conditions:

1. It should be in the **BCNF**.
2. And, the table should not have any **Multi-valued dependency**

Person	Mobile	Food_Likes
Maresh	9893/9424	Burger / pizza
Ramesh	9191	Pizza

WHAT IS MULTI-VALUED DEPENDENCY?

A table is said to have multi-valued dependency, if the following conditions are true,

For a dependency $A \twoheadrightarrow B$,

1. if for a single value of A, multiple value of B exists, then the table may have multi-valued dependency.
2. Also, a table should have at-least 3 columns for it to have a multi-valued dependency.
3. And, for a relation $R(A,B,C)$, if there is a multi-valued dependency between, A and B, then B and C should be independent of each other.

If all these conditions are true for any relation(table), it is said to have multi-valued dependency.

Example:

Relation : R		
STU_ID	STU_SKILLS	STU_HOBBIES
1	C, JAVA	Cricket, FootBall
2	JAVA	Cricket
3	C	FootBall
Multivalued FD: {STU_SKILLS, STU_HOBBIES} & stu_skills & stu_hobbies are independent		
STU_ID->STU_SKILLS, STU_ID->STU_HOBBIES,		

Relation R, Decomposed into R1 & R2		
Relation : R1		
STU_ID	STU_SKILLS	
1	C	
1	JAVA	
2	JAVA	
3	C	
Relation : R2		
STU_ID	STU_HOBBIES	
1	Cricket	
1	FootBall	
2	Cricket	
3	FootBall	

5. FIFTH NORMAL FORM (5NF) OR PJNF(PROJECTJOINNf)

For a table to be in the 5NF, it must satisfy two conditions:

For a table to satisfy the Fifth Normal Form, it should satisfy the following two conditions:

1. It should be in the 4NF.
2. It should not have **Join dependency** (i.e., Loss less decomposition, no extra record to be inserted and no information loss. Then we can say that relation in 5NF

Example:

Relation : R1	
STU_ID	STU_SKILLS
1	C
1	JAVA
2	JAVA
3	C

Relation : R2	
STU_ID	STU_HOBBIES
1	Cricket
1	FootBall
2	Cricket
3	FootBall

R: R1 joins R2, No loss of information and no extra information is added.

Relation : R		
STU_ID	STU_SKILLS	STU_HOBBIES
1	C	FootBall
1	JAVA	Cricket
2	JAVA	Cricket
3	C	FootBall